

MapReduce → Spark

Du traitement batch distribué au moteur de calcul moderne



Cours 1h30 • TP 2h • Projet fil rouge DORA

Mapper • Shuffle • Reducer • DataFrame • Lazy Evaluation • Spark on YARN

Where Are We?

Trois responsabilités distinctes dans la plateforme Hadoop

HDFS

Where data lives

Stockage distribué

YARN

Who gets resources

CPU · Mémoire · Queues

PROCESSING

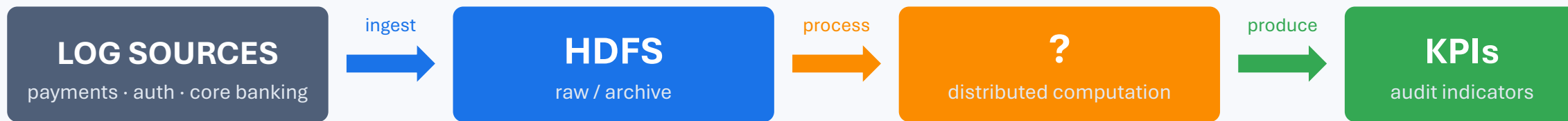
What computation runs

MapReduce puis Spark

Aujourd'hui : comprendre le traitement distribué au-dessus de HDFS et YARN.

From Data to Processing

Cas fil rouge : plusieurs années de logs techniques DORA



Volume

Plusieurs To de logs, répartis sur plusieurs DataNodes.

Parallelism

Exécuter plusieurs tâches simultanément.

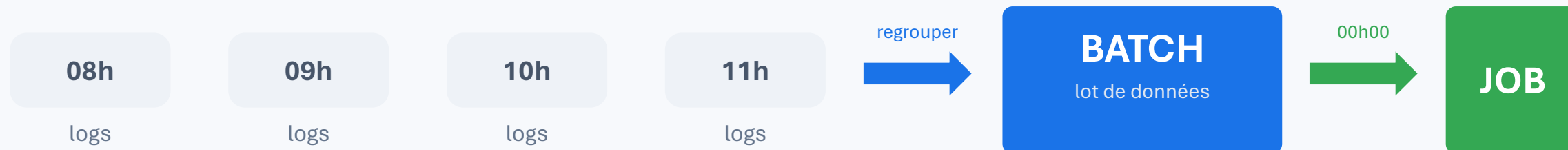
Resilience

Rejouer une tâche sans relancer tout le traitement.

Le problème n'est plus seulement de stocker les données : il faut distribuer le calcul.

What Is a Batch?

Un lot de données ou de tâches traité en une seule fois



Exemples

Paie mensuelle · rapport nocturne ·
archivage quotidien

Objectif

Traiter un ensemble complet avec un débit
élevé.

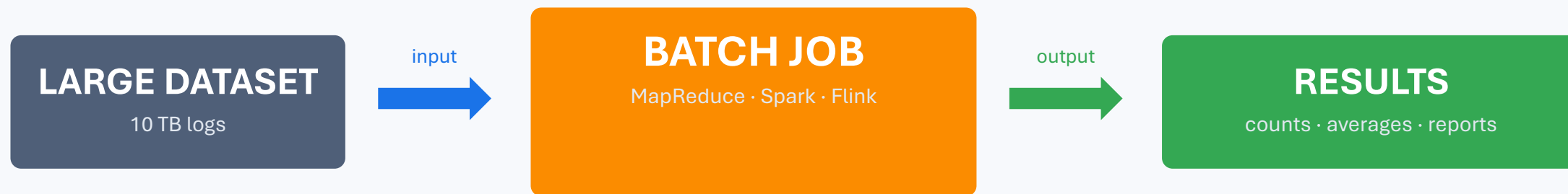
Compromis

La réponse n'est pas nécessairement
immédiate.

Batch = un lot. Le traitement batch privilégie le débit global plutôt que la réponse instantanée.

Batch Processing

Accumuler, traiter, produire un résultat



Historical Hadoop model

MapReduce est conçu pour le traitement batch distribué.

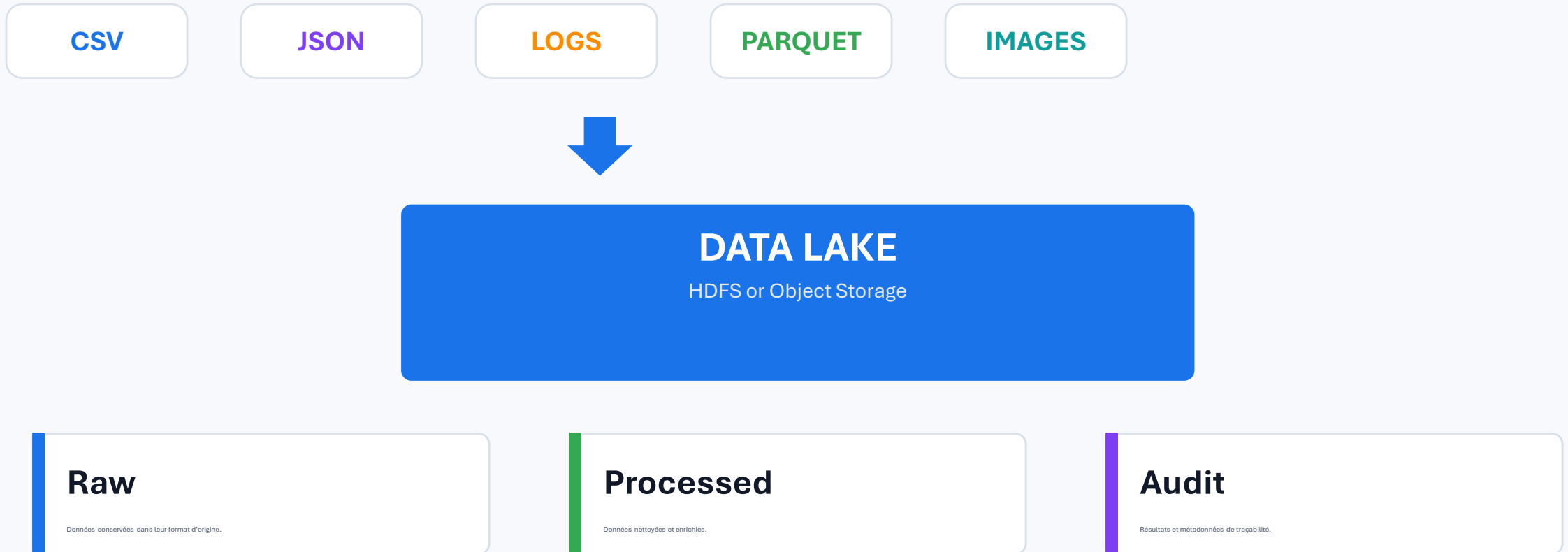
Modern engines

Spark conserve le batch, avec un modèle d'exécution plus général.

MapReduce est historiquement le moteur batch emblématique de Hadoop.

What Is a Data Lake?

Un stockage centralisé pour des données de formats variés



Dans notre architecture, HDFS fournit le stockage du Data Lake Hadoop.

Data Lake vs Data Warehouse

Deux finalités complémentaires



DATA LAKE

Données brutes ou peu transformées
Tous formats
Data Engineering / ML
Schema-on-read fréquent

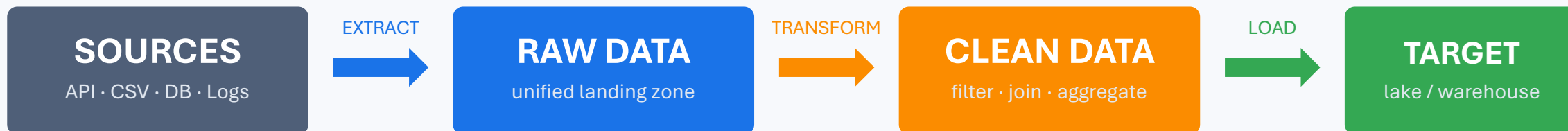
DATA WAREHOUSE

Données nettoyées et modélisées
Principalement structurées
BI / reporting / KPI
Schema-on-write fréquent

Le Data Lake conserve ; le Data Warehouse organise les données pour l'analyse métier.

ETL - Extract, Transform, Load

Une chaîne de préparation des données



Extract

Lire les données depuis les systèmes sources.

Transform

Nettoyer, filtrer, joindre et agréger.

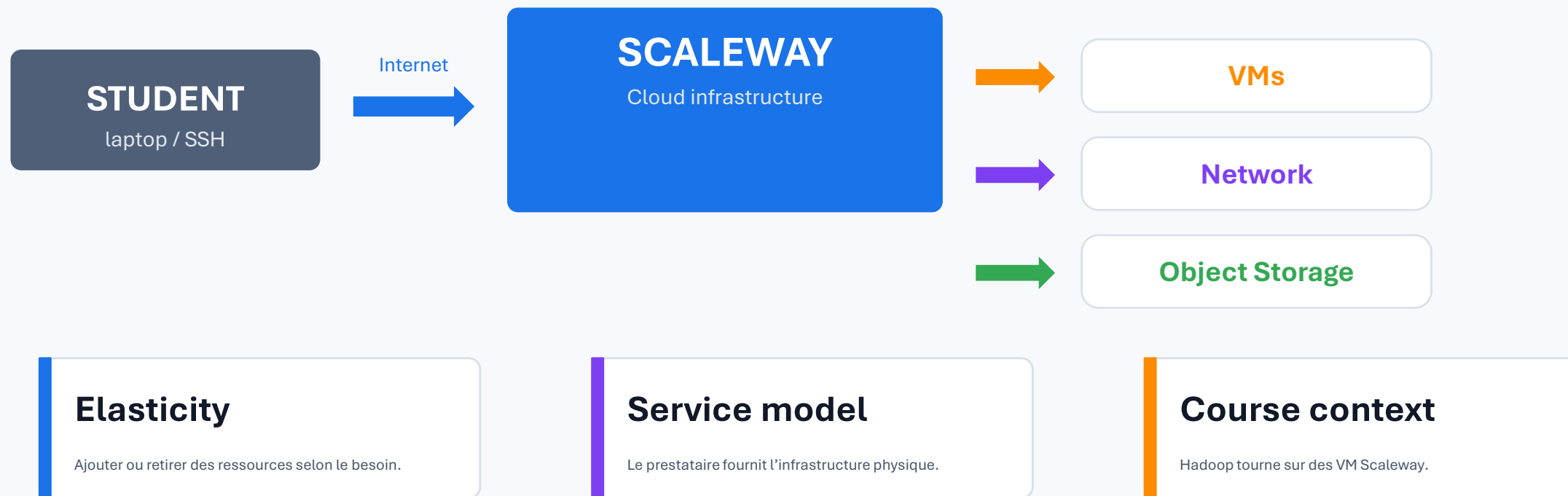
Load

Écrire dans HDFS, Parquet ou un entrepôt.

MapReduce et Spark sont des moteurs capables d'exécuter les transformations à grande échelle.

Cloud Computing

Consommer des ressources informatiques sous forme de services



Hadoop et Cloud Computing sont deux concepts distincts : notre cluster Hadoop est hébergé dans le Cloud.

In-Memory Computing

Réutiliser des données en mémoire pour limiter certaines I/O

DISK-ORIENTED WORKFLOW

Job 1
↓ write/read
Job 2
↓ write/read
Job 3

REUSE IN MEMORY

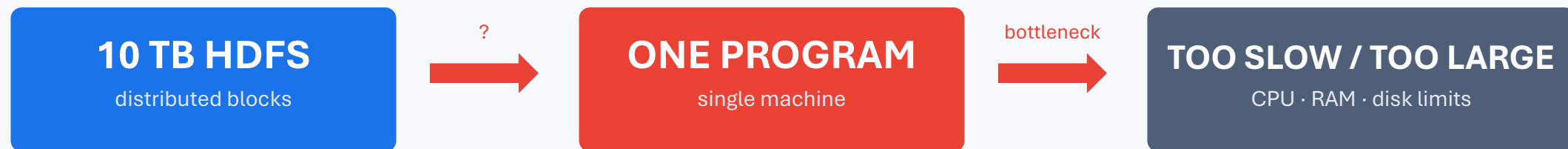
Load once
↓
Step 1 → Step 2 → Step 3
↓
Result

Spark can cache - and may also spill to disk

Spark n'est pas simplement « MapReduce en RAM » : son modèle d'exécution est plus général.

How Do We Process 10 TB of Logs?

Un programme classique ne suffit plus



Divide

Découper les données et le travail.

Parallelize

Exécuter plusieurs tâches simultanément.

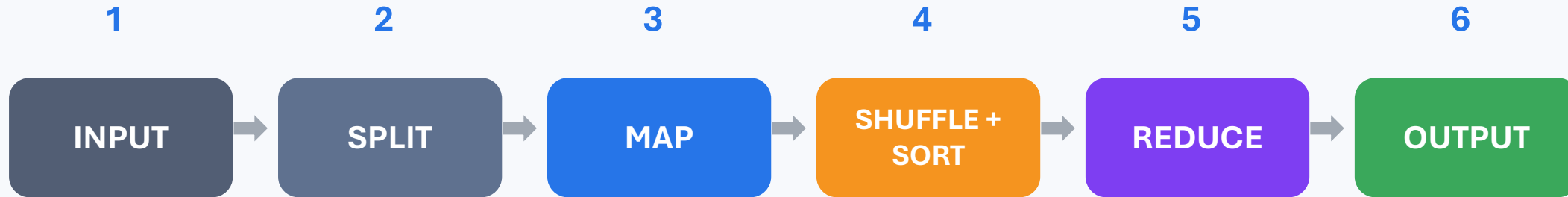
Aggregate

Regrouper les résultats intermédiaires.

MapReduce formalise cette stratégie : diviser, traiter en parallèle, regrouper.

MapReduce in One Sentence

Read → Split → Process in Parallel → Group → Aggregate



Le modèle enchaîne : données d'entrée → découpage en InputSplits → traitement parallèle → regroupement → agrégation finale.

Everything Becomes Key / Value

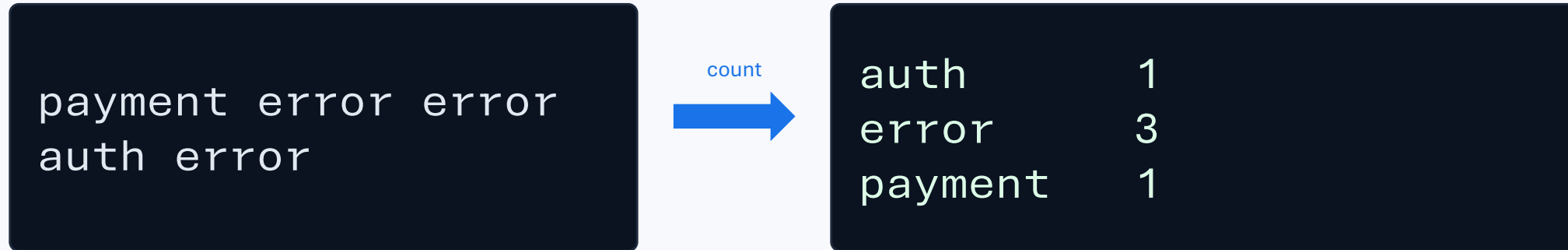
Le contrat de données du pipeline MapReduce



Les types peuvent changer entre l'entrée, les données intermédiaires et la sortie.

Our First Problem: WordCount

Un exemple simple qui révèle tout le pipeline



Question 1

Comment émettre un comptage local ?

Question 2

Comment réunir les occurrences du même mot ?

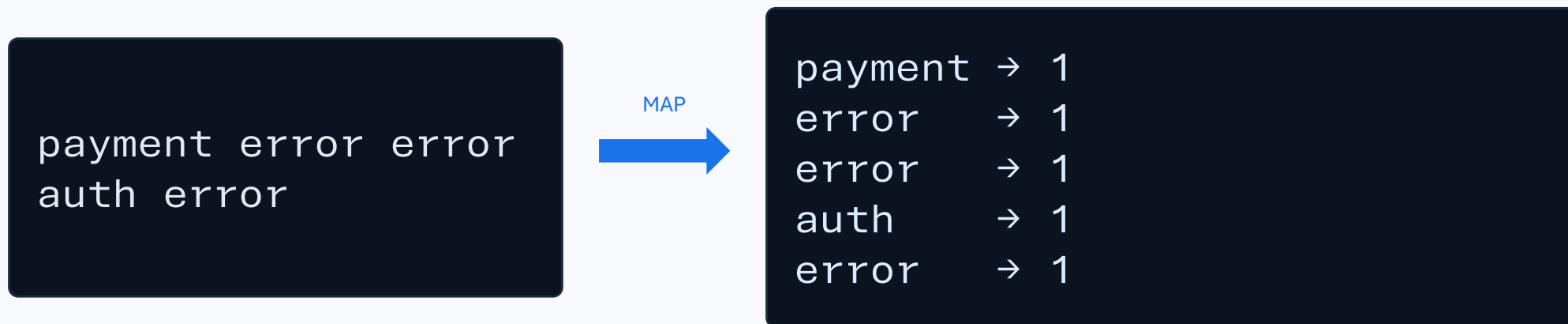
Question 3

Comment produire le total final ?

WordCount sert de fil conducteur pour Mapper, Shuffle, Sort et Reducer.

Step 1 - Mapper

Traiter chaque enregistrement indépendamment



Input

Une paire clé / valeur.

Logic

Une fonction appliquée record par record.

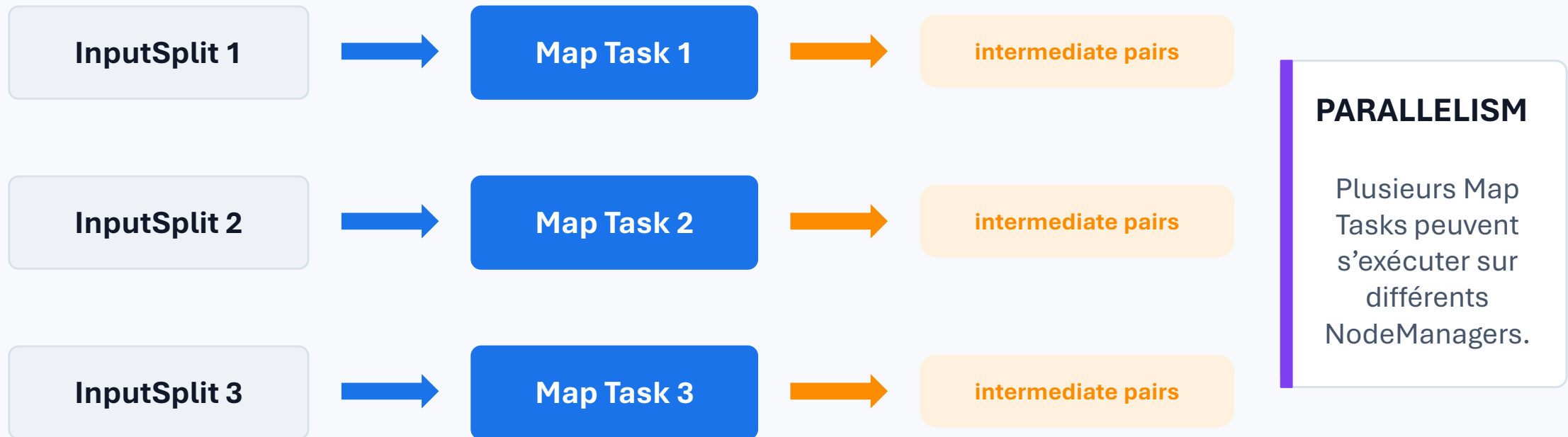
Output

Zéro, une ou plusieurs paires intermédiaires.

Le Mapper ne calcule pas encore le total global : il produit des paires intermédiaires.

Many Mappers Work in Parallel

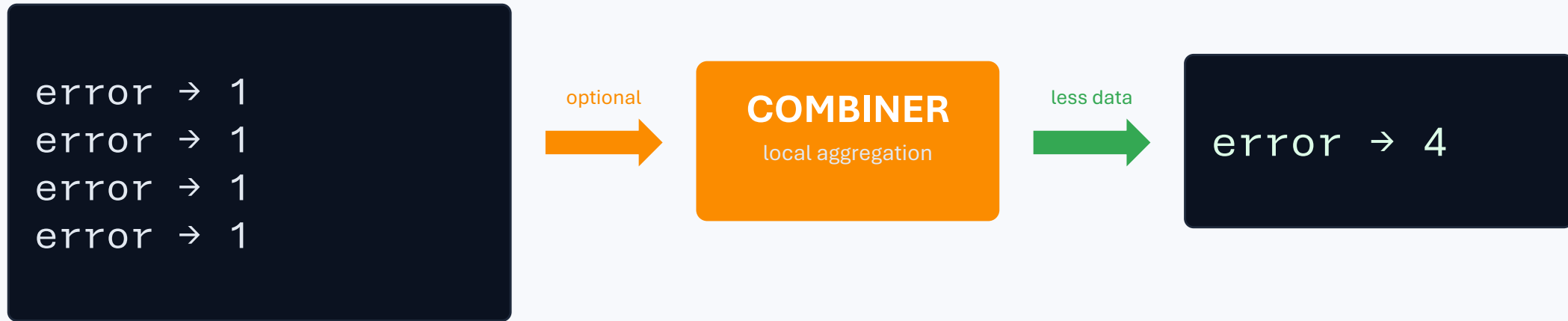
Le parallélisme commence avec les InputSplits



Le nombre de Map Tasks dépend principalement du découpage des entrées, pas du nombre de Reducers.

Combiner - Local Aggregation

Réduire le volume avant le transfert réseau



Optional

Le framework peut l'exécuter zéro, une ou plusieurs fois.

Local

Il agit sur la sortie d'un Mapper, avant le Shuffle.

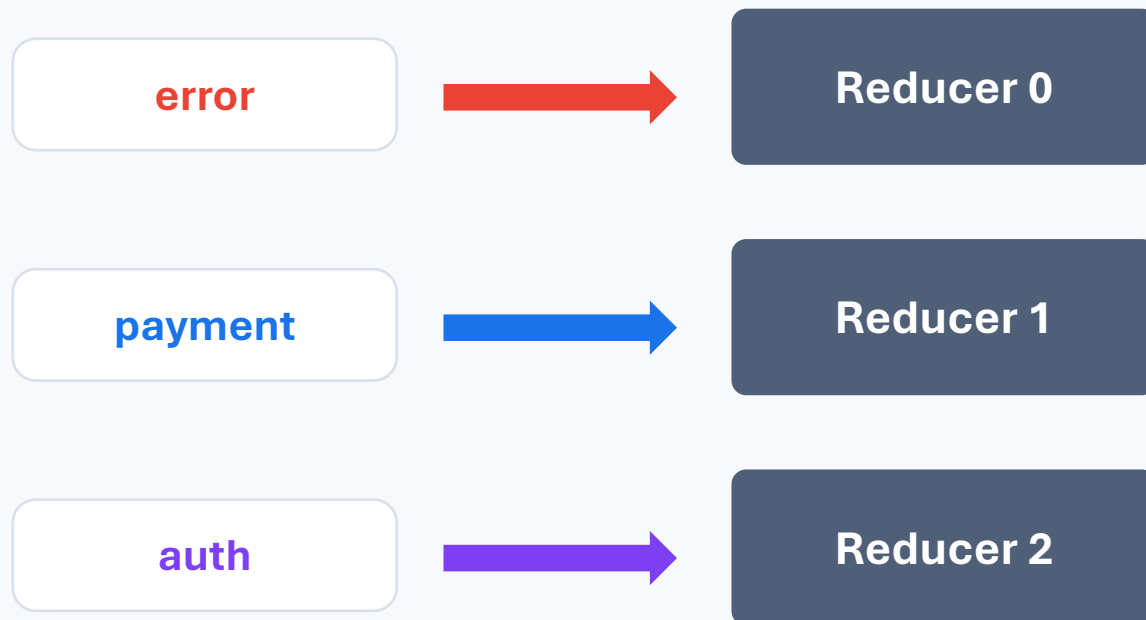
Safe operations

L'opération doit rester correcte malgré les regroupements partiels.

Combiner = optimisation facultative, pas garantie fonctionnelle du résultat.

Partitioner - Who Gets Which Key?

Diriger toutes les valeurs d'une clé vers le même Reducer



```
partition = hash(key)
            %
numberOfReducers
```

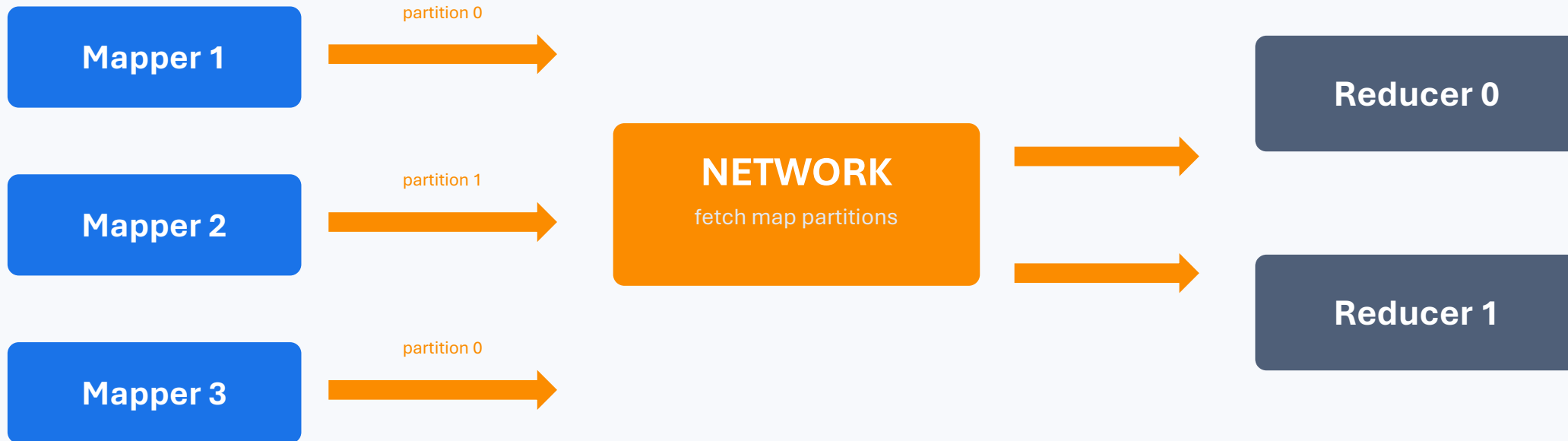
Invariant

Une même clé doit toujours être envoyée vers la même partition.

Le nombre de partitions correspond au nombre de Reduce Tasks.

Shuffle - Data Moves Across the Cluster

Redistribution réseau des sorties Mapper



Network cost

Le Shuffle transfère les partitions vers les Reducers.

Synchronization

Les Reducers récupèrent les sorties de tous les Mappers concernés.

Optimization target

Compression, Combiner et bon partitionnement réduisent le coût.

Le Shuffle est souvent l'étape la plus coûteuse d'un job MapReduce.

Sort & Group

Rassembler toutes les valeurs associées à une clé

```
error    1
auth     1
error    1
payment  1
error    1
```

sort + group



```
auth     → [1]
error    → [1, 1, 1]
payment  → [1]
```

Sort

Les clés sont ordonnées dans chaque partition.

Group

Les valeurs partageant une clé sont regroupées.

Reducer input

<key, iterable(values)>

Le Reducer reçoit une clé et l'ensemble de ses valeurs intermédiaires.

Reducer

Agrégation finale par clé



Input

Une clé + toutes ses valeurs.

Function

Somme, moyenne, min/max, logique métier.

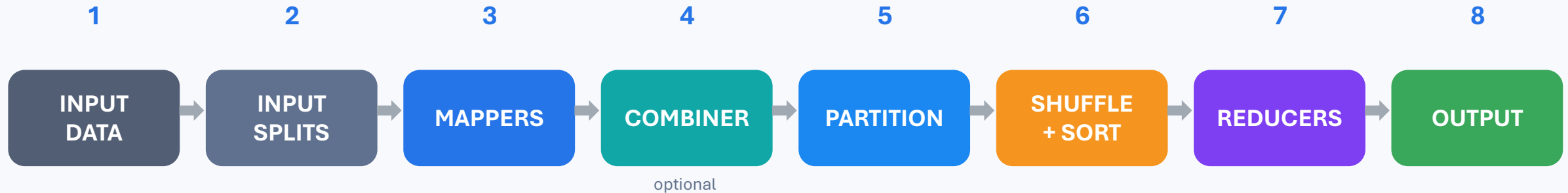
Output

Résultats écrits dans le FileSystem.

Un job peut avoir plusieurs Reducers - ou aucun Reducer pour un map-only job.

The Complete MapReduce Pipeline

Une vue de synthèse - chaque étape a un rôle précis

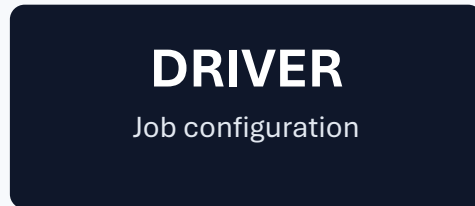


Input	Original records read from HDFS or another filesystem.
Split	Logical input partitions used to create Map Tasks.
Map side	Map, optional combine and partition happen before the network exchange.
Shuffle + Sort	Reducers fetch map outputs, then merge, sort and group them by key.
Reduce side	Reducers aggregate and write the final output files.

Le découpage en InputSplits précède les Mappers : c'est lui qui détermine le parallélisme côté Map.

The Driver

Configurer et soumettre le job



Mapper class

Reducer class

Combiner / Partitioner

Input / Output paths

Key / Value types

Number of reducers

```
job.setMapperClass(...); job.setReducerClass(...);  
job.waitForCompletion(true);
```

Le Driver décrit le job ; YARN fournit les ressources nécessaires à son exécution.

One Job, Many Tasks

Un job est décomposé en unités d'exécution parallèles



Job

Le traitement logique complet.

Task

Une unité de travail planifiée par YARN.

Attempt

Une exécution particulière d'une tâche.

Une Task peut avoir plusieurs Attempts en cas d'échec ou de relance.

MapReduce + HDFS + YARN

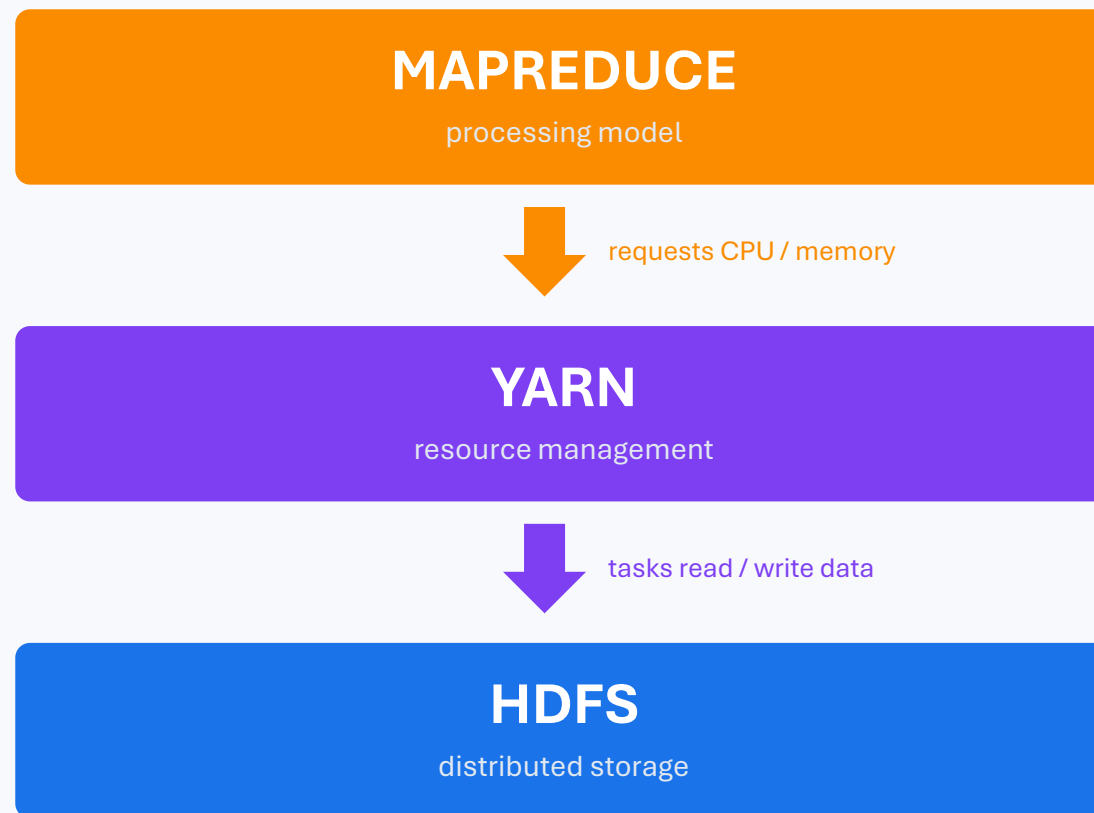
Trois couches, trois responsabilités

KEY IDEA

MapReduce n'est pas un système de stockage.

YARN n'est pas un moteur de traitement.

HDFS n'exécute pas les calculs.



MapReduce utilise YARN pour les ressources et HDFS pour les entrées et sorties.

Data Locality

Rapprocher le calcul des blocs HDFS

PREFERRED

DataNode 1
Block A
+
Map Task A

Data stays local

LESS EFFICIENT

DataNode 1: Block A

network transfer →

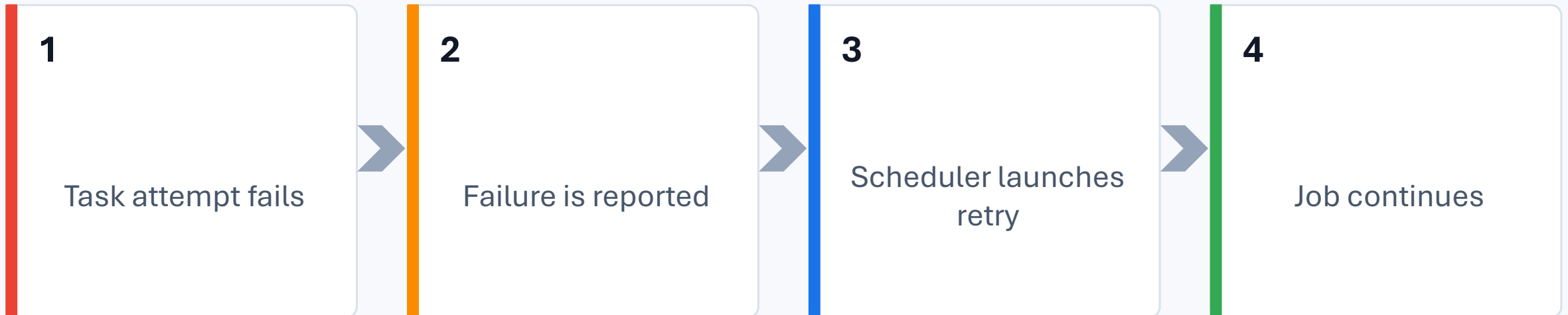
NodeManager 3: Map Task A

locality reduces network traffic

YARN et MapReduce essaient d'exécuter les Map Tasks près des données lorsque c'est possible.

What Happens When a Task Fails?

La tolérance aux pannes s'applique aussi au calcul



Une tâche peut être rejouée sans recommencer tout le job - tant que les entrées restent disponibles.

MapReduce Changed Distributed Computing

Pourquoi ce modèle reste fondamental à étudier

Simple model

Map puis Reduce : une structure claire pour paralléliser.

Scale-out

Des milliers de tâches sur de nombreuses machines.

Fault tolerance

Relance de tâches et données répliquées.

Data locality

Déplacer le calcul plutôt que les données.

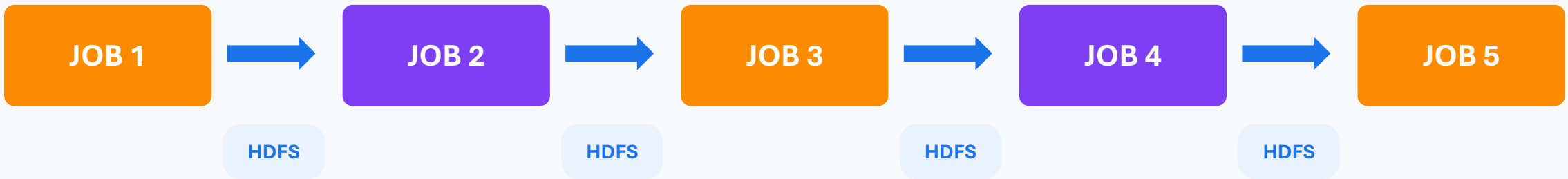
Batch throughput

Efficace pour de grands traitements séquentiels.

MapReduce est aujourd'hui surtout étudié pour comprendre les fondements du calcul distribué.

But What About Complex Pipelines?

Un traitement réel comporte souvent plusieurs étapes



More code

Chaque étape devient un nouveau job à configurer.

More I/O

Les frontières de jobs nécessitent des matérialisations.

More latency

Le pipeline complet devient lourd à exécuter.

MapReduce est simple pour un job ; l'enchaînement de nombreux jobs devient complexe.

Disk I/O Becomes Expensive

Le coût des matérialisations intermédiaires



Storage bandwidth

Les lectures et écritures consomment du débit disque.

Serialization

Les données doivent être encodées et décodées.

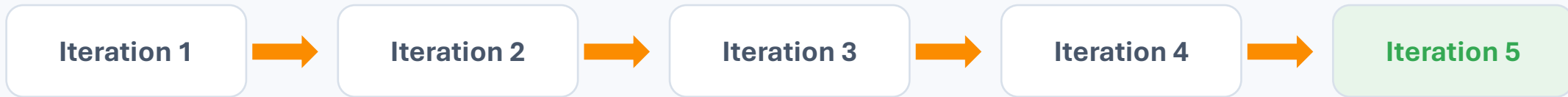
Job latency

Chaque étape ajoute du démarrage et de la coordination.

Le problème n'est pas « le disque est toujours mauvais », mais le nombre d'étapes intermédiaires imposées.

What About Iterative Algorithms?

Répéter le même calcul sur les mêmes données



... jusqu'à convergence

Machine Learning

Réutilisation répétée du dataset.

Graph processing

Propagation itérative entre sommets.

Interactive analysis

Plusieurs requêtes successives sur les mêmes données.

Les workloads itératifs bénéficient d'un moteur capable de réutiliser des données et d'optimiser plusieurs étapes.

Can We Build a Better Execution Model?

Décrire tout le pipeline avant de l'exécuter

SEQUENCE OF JOBS

Job 1 → storage

Job 2 → storage

Job 3 → storage

Each job optimized alone

EXECUTION GRAPH

filter → select → groupBy → write

One logical plan

Optimized before execution

Spark permet d'exprimer une chaîne de transformations puis de construire un plan d'exécution global.

Enter Apache Spark

A general-purpose distributed processing engine

APACHE SPARK

distributed processing engine

Expressive APIs

SQL, DataFrames, RDDs, Python, Scala, Java.

Execution plans

Transformations organisées en jobs, stages et tasks.

Multiple workloads

Batch, SQL, streaming and machine learning.

Spark processes data - it does not store it

Spark est un moteur de calcul : il lit les données depuis HDFS ou d'autres systèmes de stockage.

HDFS vs YARN vs Spark

Un rôle précis pour chaque composant

HDFS

STORAGE

Fichiers distribués
Blocs et réplication
Lecture / écriture

YARN

RESOURCES

CPU et mémoire
Containers
Scheduling et queues

SPARK

PROCESSING

DataFrames
Execution plan
Jobs, stages, tasks

Spark peut fonctionner sur YARN et lire ou écrire dans HDFS.

MapReduce vs Spark

Deux modèles de traitement distribué

DIMENSION	MAPREDUCE	SPARK
Programming model	Mapper + Reducer	Graph of transformations
Pipeline	Multiple jobs	One optimized plan
API	Lower level	More expressive
Intermediate reuse	Limited across jobs	Cache / reuse possible
Best fit	Stable batch jobs	Complex data pipelines

Spark ne « rend pas MapReduce faux » : il généralise le modèle pour des pipelines plus complexes.

SparkSession

Le point d'entrée principal des APIs structurées Spark



Read

`spark.read.csv(...)`

Query

`spark.sql(...)`

Context

`spark.sparkContext`

Dans le shell PySpark du TP, la variable `spark` est déjà disponible.

DataFrame

Une table distribuée avec un schéma

timestamp	application	level	response_time_ms
10:01	payment-api	INFO	120
10:02	auth-service	ERROR	850
10:03	core-banking	WARN	990

```
df.filter(...)  
df.select(...)  
df.groupBy(...)  
df.orderBy(...)
```

Schema

Noms et types des colonnes.

Distributed

Les lignes sont réparties en partitions.

Declarative

On décrit le résultat attendu.

Le TP utilisera les DataFrames PySpark pour analyser les logs DORA.

Transformation vs Action

Décrire un calcul ou déclencher son exécution

TRANSFORMATIONS

filter()
select()
groupBy()
orderBy()

Return a new DataFrame

ACTIONS

show()
count()
collect()
write.parquet()

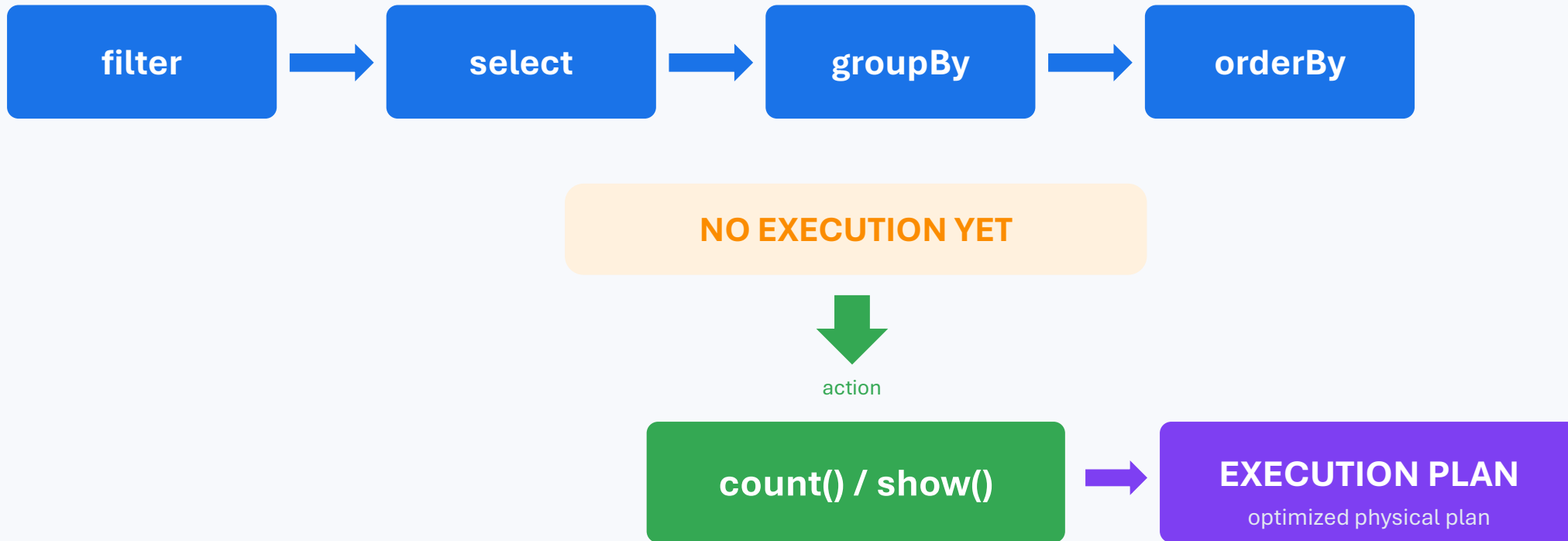
Trigger computation

Transformation builds - Action executes

Une transformation décrit un nouveau DataFrame ; une action demande un résultat concret.

Lazy Evaluation

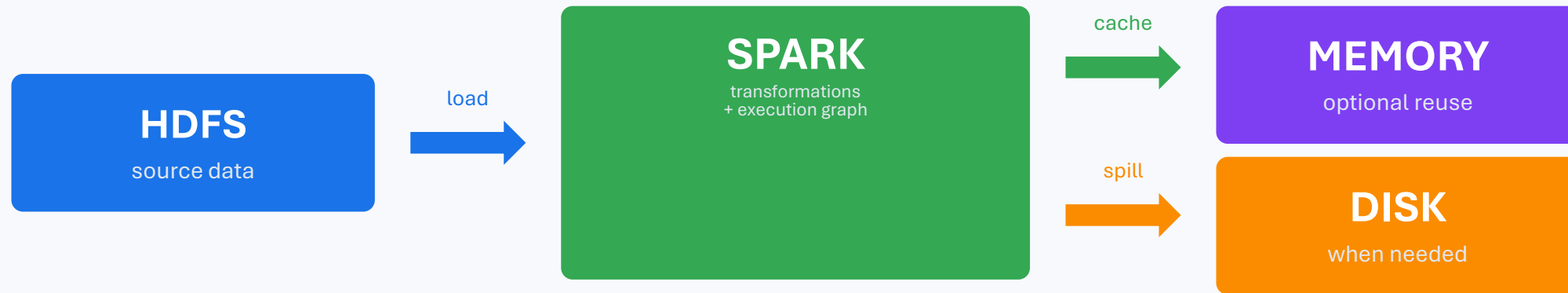
Spark construit le plan avant d'exécuter



Connaître plusieurs transformations permet à Spark d'optimiser le plan avant l'exécution.

From In-Memory Computing to Spark

La mémoire comme optimisation contrôlée



cache()

Demander explicitement la réutilisation.

Memory pressure

Les données peuvent être évincées ou recomputées.

Spill

Certaines opérations écrivent sur disque.

Spark utilise la mémoire intelligemment, mais reste un moteur hybride mémoire + disque.

Spark on YARN

Le TP utilise le mode client depuis le gateway



Driver

En deploy-mode client, le driver reste sur le gateway.

YARN

Alloue les containers des executors.

Executors

Exécutent les tasks et accèdent à HDFS.

Spark exploite YARN comme cluster manager et HDFS comme stockage distribué.

TP04 - Introduction to Spark

Du premier DataFrame au spark-submit sur YARN

1 · Prepare

Logs CSV → HDFS

2 · Explore

PySpark + SparkSession

3 · Transform

filter · select · groupBy

4 · Execute

actions · explain()

5 · Persist

Parquet → HDFS

6 · Operate

spark-submit · YARN · UIs

Final comparison: WordCount MapReduce vs Spark DataFrame

Objectif du TP : observer le passage du modèle MapReduce à un pipeline Spark exécuté sur YARN.